



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)

Section : Section 1-2

Task : Project II

Due : 19th January 2026 (10%)

Name	Matric No.
Cheryl Cheong Kah Voon	A23CS0060
Lau Yee Wen	A23CS0099
Lee Yin Shen	A23CS0236
Poh Lok Yee	A23CS0262

Project Overview

This mini project enables students to design and implement a **NoSQL database backend** using **MongoDB**. Students are expected to apply the concepts learned in the NoSQL and MongoDB topics by modelling a selected domain, storing data in collections, and performing CRUD operations, indexing, and aggregation workflows.

***Note:** This project focuses on **MongoDB backend development only**. No front-end or UI is required.*

Through this project, students will:

- Apply NoSQL principles and the **Document Data Model**.
- Demonstrate CRUD operations in MongoDB (insert, find, update, delete).
- Use MongoDB **query operators, projections, and update operators**.
- Implement **indexing** to improve performance.
- Build **aggregation pipelines** for analytics.
- Analyse NoSQL security considerations and design challenges.
- Provide output samples through a structured report and presentation.

By the end of this project, each group (3–4 members) should be able to demonstrate a working NoSQL backend capable of supporting common non-relational workloads such as activity management, content systems, event systems, logging systems, or any domain of choice.

Table of Contents

1.0 Introduction	2
1.1 Description of Domain	2
1.2 Objectives	2
1.3 Team Members and Roles	2
2.0 NoSQL Data Model	3
2.1 Collections	3
2.2 JSON Documents Example	3
2.3 Embedding and Referencing	4
2.4 Data Types Used	4
3.0 Basic MongoDB Operations (CRUD)	5
3.1 Insert Operations (Create)	5
3.2 Query Operations (Read)	6
3.3 Update Operations (Update)	15
3.4 Delete Operations (Delete)	18
4.0 Advanced Operations	19
4.1 Index Creation	19
4.2 Aggregation Pipelines	19
4.3 Sorting Demonstration	20
4.4 Explanation of Performance Improvements	23
4.5 Challenges Encountered	23
5.0 Security and Limitations	24
6.0 Teamwork	25
7.0 Conclusion	27

1.0 Introduction

1.1 Description of Domain

The chosen domain for this project is a simple event booking system. It is a platform designed to allow participants to browse, register and manage their attendance at events organized by companies or brands. The system mainly tracks participant information and bookings, event and organizers details which provide an efficient way to handle registrations and monitor participant activity.

1.2 Objectives

The objectives of this project are:

1. To design a NoSQL database backend by using MongoDB that accurately models the event booking system
2. To implement CRUD operations for managing participants, events and bookings.
3. To apply indexing and aggregation pipelines for improved query performance
4. To analyse NoSQL security considerations, access control, and consistency trade-offs

1.3 Team Members and Roles

This project team consists of four members, each responsible for a specific aspect of the system.

Name	Role	Responsibilities
Poh Lok Yee	Database Designer	Designing collections and creating sample documents.
Lee Yin Shen	CRUD Specialist	Implementing insertion, querying, updating, and deletion of documents.
Cheryl Cheong Kah Voon	Data Analyst	Implementing insertion, querying, updating, and deletion of documents.
Lau Yee Wen	Performance & Security Specialist	Evaluating access controls, injection risks and consistency considerations

2.0 NoSQL Data Model

2.1 Collections

The Event Booking System uses four primary collections as shown at the figure and table below.



Collection	Purpose
bookings	Tracks participant bookings for events and includes payment details along with booking status.
events	Contains details of event, including title, organizer, date, price and tags.
organizers	Stores information about event organizers, including name, industry type, country, and contact details.
participants	Represents users who register for events, including their personal information and the events they have booked.

2.2 JSON Documents Example

Collection	JSON Document	Collection	JSON Document
bookings	<pre>{ "_id": ObjectId('695adc92f084f228b404f080'), "participantId": ObjectId('695acc20f084f228b404f05b'), "eventId": ObjectId('695abc9cf084f228b404f029'), "bookingDate": 2025-12-01T00:00:00.000Z, "status": 'Confirmed', "payment": { "method": 'Credit Card', "amount": 100, "paidAt": 2025-12-01T01:05:00.000Z } }</pre>	organizers	<pre>{ "_id": ObjectId('695ab30df084f228b404f025'), "name": 'Grab', "industry": 'Technology', "country": 'Malaysia', "contactEmail": 'grab@gmail.com', "contactNumber": '03-1111 5678' }</pre>
events	<pre>{ "_id": ObjectId('695abc9cf084f228b404f029'), "title": 'Grab Tech Summit 2026', "category": 'Technology', "organizerId": ObjectId('695ab30df084f228b404f025'), "location": 'Kuala Lumpur', "eventDate": 2026-06-15T00:00:00.000Z, "price": 100, "capacity": 100, "tags": ['Technology', 'Innovation', 'Networking'] }</pre>	participants	<pre>{ "_id": ObjectId('695acc20f084f228b404f05b'), "firstName": 'Lok Yee', "lastName": 'Poh', "email": 'polok@gmail.com', "phone": '011-10026148', "gender": 'Female', "age": 22, "address": 'Kuala Lumpur', "bookedEventIds": ObjectId('695abc9cf084f228b404f029') }</pre>

2.3 Embedding and Referencing

Embedding	Referencing
- Stores related data directly inside a document - Suitable when data closely related Used for the payment information within each booking, keeping details in a single document for easier access. <pre> { _id: ObjectId('695adc92f084f228b404f080'), participantId: ObjectId('695acc20f084f228b404f05b'), eventId: ObjectId('695abc9cf084f228b404f029'), bookingDate: 2025-12-01T00:00:00.000Z, status: 'Confirmed', payment: { method: 'Credit Card', amount: 100, paidAt: 2025-12-01T01:05:00.000Z } }</pre>	- Links documents across collections using unique identifiers - Suitable when data is large, shared or frequently updated Applied in Bookings collection using participantId and eventId to link each booking to a participant and an event <pre> { _id: ObjectId('695adc92f084f228b404f080'), participantId: ObjectId('695acc20f084f228b404f05b'), eventId: ObjectId('695abc9cf084f228b404f029'), bookingDate: 2025-12-01T00:00:00.000Z, status: 'Confirmed', payment: { method: 'Credit Card', amount: 100, paidAt: 2025-12-01T01:05:00.000Z } }</pre>

2.4 Data Types Used

Data Types	Data	Examples
String	Name, email, phone, gender, address, industry, country, title, category, location, status, method	<pre> name: 'Grab', industry: 'Technology', country: 'Malaysia', contactEmail: 'grab@gmail.com', contactNumber: '03-1111 5678'</pre>
Number	age, price, capacity, amount	<pre> price: 100, capacity: 100,</pre>
Date	eventDate, bookingDate, paidAt	<pre> eventDate: 2026-06-15T00:00:00.000Z,</pre>
Array	tags in events and bookedEventIds in participants	<pre> tags: ['Technology', 'Innovation', 'Networking']</pre>
Object	Embedded payment details in bookings	<pre> payment: { method: 'Credit Card', amount: 100, paidAt: 2025-12-01T01:05:00.000Z }</pre>

3.0 Basic MongoDB Operations (CRUD)

3.1 Insert Operations (Create)

3.1.1 insertOne()

Use the insertOne() command to add a new participant (Ricky) into the participants collection.

```
> db.participants.insertOne({
  "firstName": "Ricky",
  "lastName": "Tan",
  "email": "ricky@gmail.com",
  "phone": "012-9998887",
  "gender": "Male",
  "age": 21,
  "address": "Selangor"
})
< {
  acknowledged: true,
  insertedId: ObjectId('695c74edcb0ca891e1a6feea')
}
```

```
< {
  acknowledged: true,
  insertedId: ObjectId('695c74edcb0ca891e1a6feea')
}
```

```
_id: ObjectId('695c74edcb0ca891e1a6feea')
firstName : "Ricky"
lastName : "Tan"
email : "ricky@gmail.com"
phone : "012-9998887"
gender : "Male"
age : 21
address : "Selangor"
```

3.1.2 insertMany()

Use the insertMany() command to add multiple events into the events collection.

```
> db.events.insertMany([
  {
    "title": "AI & Future Tech Symposium",
    "category": "Technology",
    "organizerID": ObjectId("695ab30df084f228b404f025"),
    "location": "Kuala Lumpur",
    "eventDate": ISODate("2026-07-20T09:00:00Z"),
    "price": 120,
    "capacity": 200,
    "tags": ["AI", "Future Tech", "Symposium"]
  },
  {
    "title": "Digital Marketing Mastery 2026",
    "category": "Business",
    "organizerID": ObjectId("695ab30df084f228b404f026"),
    "location": "Penang",
    "eventDate": ISODate("2026-08-15T10:00:00Z"),
    "price": 45,
    "capacity": 80,
    "tags": ["Marketing", "Digital", "Workshop"]
  },
  {
    "title": "Cloud Computing Bootcamp",
    "category": "Technology",
    "organizerID": ObjectId("695ab30df084f228b404f025"),
    "location": "Online",
    "eventDate": ISODate("2026-09-05T08:00:00Z"),
    "price": 60,
    "capacity": 500,
    "tags": ["Cloud", "Coding", "Education"]
  }
])
```

```
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('695f007419a4856217524b65'),
    '1': ObjectId('695f007419a4856217524b66'),
    '2': ObjectId('695f007419a4856217524b67')
  }
}
```

```
_id: ObjectId('695c7bb0cb0ca891e1a6feee')
title: "AI & Future Tech Symposium"
category: "Technology"
organizerID: ObjectId('695ab30df084f228b404f025')
location: "Kuala Lumpur"
eventDate: 2026-07-20T09:00:00.000+00:00
price: 120
capacity: 200
tags: Array (3)
```

```
_id: ObjectId('695c7bb0cb0ca891e1a6feef')
title: "Digital Marketing Mastery 2026"
category: "Business"
organizerID: ObjectId('695ab30df084f228b404f026')
location: "Penang"
eventDate: 2026-08-15T10:00:00.000+00:00
price: 45
capacity: 80
tags: Array (3)
```

```
_id: ObjectId('695c7bb0cb0ca891e1a6fef0')
title: "Cloud Computing Bootcamp"
category: "Technology"
organizerID: ObjectId('695ab30df084f228b404f025')
location: "Online"
eventDate: 2026-09-05T08:00:00.000+00:00
price: 60
capacity: 500
tags: Array (3)
```

3.2 Query Operations (Read)

3.2.1 find()

Use the `find()` command to retrieve the info of all participants who is female.

<pre>> db.participants.find({ "gender": "Female" })</pre>	<pre>< { _id: ObjectId('695acc20f084f228b404f05b'), firstName: 'Lok Yee', lastName: 'Poh', email: 'polok@gmail.com', phone: '011-10026148', gender: 'Female', age: 22, address: 'Kuala Lumpur', bookedEventIds: ObjectId('695abc9cf084f228b404f029') } { _id: ObjectId('695acc20f084f228b404f05c'), firstName: 'Yee Wen', lastName: 'Lau', email: 'lyw@gmail.com', phone: '012-3456789', gender: 'Female', age: 25, address: 'Kuala Lumpur', bookedEventIds: ObjectId('695abc9cf084f228b404f02a') } { _id: ObjectId('695acc20f084f228b404f05e'), firstName: 'Heng', lastName: 'Sabrina', email: 'sab@gmail.com', phone: '014-5872613', gender: 'Female', age: 19, address: 'Melaka', bookedEventIds: ObjectId('695abc9cf084f228b404f02c') }</pre>
--	--

3.2.2 findOne()

Use findOne() command to find the first participant who gender is female.

<pre>> db.participants.findOne({ "gender": "Female" })</pre>	<pre>< { _id: ObjectId('695acc20f084f228b404f05b'), firstName: 'Lok Yee', lastName: 'Poh', email: 'polok@gmail.com', phone: '011-10026148', gender: 'Female', age: 22, address: 'Kuala Lumpur', bookedEventIds: ObjectId('695abc9cf084f228b404f029') }</pre>
---	---

3.2.3 find() + \$gt

Use the find() and \$gt command to retrieve the events info where the price is greater than 60.

```
> db.events.find({  
  "price": { $gt: 60 }  
})
```

```
< {  
  _id: ObjectId('695abc9cf084f228b404f029'),  
  title: 'Grab Tech Summit 2026',  
  category: 'Technology',  
  organizerID: ObjectId('695ab30df084f228b404f025'),  
  location: 'Kuala Lumpur',  
  eventDate: 2026-06-15T00:00:00.000Z,  
  price: 100,  
  capacity: 100,  
  tags: [  
    'Technology',  
    'Innovation',  
    'Networking'  
  ]  
}
```

3.2.4 find() + \$lt

Use the find() and \$lt command to retrieve the participants info where their age is less than 20.

```
> db.participants.find({  
  "age": { $lt: 20 }  
})
```

```
< {  
  _id: ObjectId('695acc20f084f228b404f05e'),  
  firstName: 'Heng',  
  lastName: 'Sabrina',  
  email: 'sab@gmail.com',  
  phone: '014-5872613',  
  gender: 'Female',  
  age: 19,  
  address: 'Melaka',  
  bookedEventIds: ObjectId('695abc9cf084f228b404f02c')  
}  
{  
  _id: ObjectId('695acc20f084f228b404f065'),  
  firstName: 'Tan',  
  lastName: 'Reynard',  
  email: 'tan@gmail.com',  
  phone: '019-3456823',  
  gender: 'Male',  
  age: 19,  
  address: 'Negeri Sembilan',  
  bookedEventIds: ObjectId('695abc9cf084f228b404f029')  
}
```

3.2.5 find() + \$eq

Use the find() and \$eq command to retrieve the events info where the category is equal to Business.

```
> db.events.find({  
  "category": { $eq: "Business" }  
})
```

```
< {  
  _id: ObjectId('695abc9cf084f228b404f02a'),  
  title: 'Lazada Online Seller Workshop',  
  category: 'Business',  
  organizerID: ObjectId('695ab30df084f228b404f027'),  
  location: 'Kuala Lumpur',  
  eventDate: 2026-05-10T00:00:00.000Z,  
  price: 50,  
  capacity: 70,  
  tags: [  
    'Business',  
    'E-Commerce',  
    'Workshop'  
  ]  
}  
{  
  _id: ObjectId('695abc9cf084f228b404f02b'),  
  title: 'Shopee E-Commerce Expo',  
  category: 'Business',  
  organizerID: ObjectId('695ab30df084f228b404f026'),  
  location: 'Johor Bahru',  
  eventDate: 2026-04-20T00:00:00.000Z,  
  price: 20,  
  capacity: 300,  
  tags: [  
    'Business',  
    'E-Commerce',  
    'Expo'  
  ]  
}
```

3.2.6 find() + \$in

Use the find() and \$in command to find the participants info where their address value is match the “Melaka”.

```
> db.participants.find({
  "address": { $in: ["Melaka"] }
})
```

```
< {
  _id: ObjectId('695acc20f084f228b404f05e'),
  firstName: 'Heng',
  lastName: 'Sabrina',
  email: 'sab@gmail.com',
  phone: '014-5872613',
  gender: 'Female',
  age: 19,
  address: 'Melaka',
  bookedEventIds: ObjectId('695abc9cf084f228b404f02c')
}
{
  _id: ObjectId('695acc20f084f228b404f066'),
  firstName: 'Chia',
  lastName: 'Brendan',
  email: 'bren@gmail.com',
  phone: '017-3456823',
  gender: 'Male',
  age: 29,
  address: 'Melaka',
  bookedEventIds: ObjectId('695abc9cf084f228b404f02b')
}
```

3.2.7 find() + \$and

Use the find() and \$and command to retrieve the events info that satisfied two condition at the same time, where location is “Kuala Lumpur”, and capacity is greater than 80.

```
> db.events.find({
  $and: [
    { "location": "Kuala Lumpur" },
    { "capacity": { $gt: 80 } }
  ]
})
```

```
< {
  _id: ObjectId('695abc9cf084f228b404f029'),
  title: 'Grab Tech Summit 2026',
  category: 'Technology',
  organizerID: ObjectId('695ab30df084f228b404f025'),
  location: 'Kuala Lumpur',
  eventDate: 2026-06-15T00:00:00.000Z,
  price: 100,
  capacity: 100,
  tags: [
    'Technology',
    'Innovation',
    'Networking'
  ]
}
{
  _id: ObjectId('695c7bb0cb0ca891e1a6fee'),
  title: 'AI & Future Tech Symposium',
  category: 'Technology',
  organizerID: ObjectId('695ab30df084f228b404f025'),
  location: 'Kuala Lumpur',
  eventDate: 2026-07-20T09:00:00.000Z,
  price: 120,
  capacity: 200,
  tags: [
    'AI',
    'Future Tech',
    'Symposium'
  ]
}
```

3.2.8 find() + \$or

Use the find() and \$or command to retrieve the events info that satisfied either one of the condition, where either category is “Technology” or price is less then 30.

```
> db.events.find({
  $or: [
    { "category": "Technology" },
    { "price": { $lt: 30 } }
  ]
})
```

```
< {
  _id: ObjectId('695abc9cf084f228b404f029'),
  title: 'Grab Tech Summit 2026',
  category: 'Technology',
  organizerID: ObjectId('695ab30df084f228b404f025'),
  location: 'Kuala Lumpur',
  eventDate: 2026-06-15T00:00:00.000Z,
  price: 100,
  capacity: 100,
  tags: [
    'Technology',
    'Innovation',
    'Networking'
  ]
}
{
  _id: ObjectId('695abc9cf084f228b404f02b'),
  title: 'Shopee E-Commerce Expo',
  category: 'Business',
  organizerID: ObjectId('695ab30df084f228b404f026'),
  location: 'Johor Bahru',
  eventDate: 2026-04-20T00:00:00.000Z,
  price: 20,
  capacity: 300,
  tags: [
    'Business',
    'E-Commerce',
    'Expo'
  ]
}
```

3.2.9 find() + Array queries

Use the find() command to query an array field to retrieve all events where the "tags" array contains the value "Technology".

```
> db.events.find({
  "tags": "Technology"
})
```

```
< {
  _id: ObjectId('695abc9cf084f228b404f029'),
  title: 'Grab Tech Summit 2026',
  category: 'Technology',
  organizerID: ObjectId('695ab30df084f228b404f025'),
  location: 'Kuala Lumpur',
  eventDate: 2026-06-15T00:00:00.000Z,
  price: 100,
  capacity: 100,
  tags: [
    'Technology',
    'Innovation',
    'Networking'
  ]
}
{
  _id: ObjectId('695abc9cf084f228b404f02c'),
  title: 'Grab Ride & Tech Conference',
  category: 'Technology',
  organizerID: ObjectId('695ab30df084f228b404f025'),
  location: 'Johor Bahru',
  eventDate: 2026-02-24T16:00:00.000Z,
  price: 50,
  capacity: 150,
  tags: [
    'Technology',
    'Conference',
    'Mobility'
  ]
}
```

3.2.10 find() + Projection

Use the find() command with projection to retrieve all events info. Use 1 to show the title and price fields and 0 to hide the `_id` field.

```
> db.events.find(
  {},
  { "title": 1, "price": 1, "_id": 0 }
)
```

```
< {
  title: 'Grab Tech Summit 2026',
  price: 100
}
{
  title: 'Lazada Online Seller Workshop',
  price: 50
}
{
  title: 'Shopee E-Commerce Expo',
  price: 20
}
{
  title: 'Grab Ride & Tech Conference',
  price: 50
}
{
  title: 'Lazada Seller Growth Seminar',
  price: 80
}
{
  title: 'AI & Future Tech Symposium',
  price: 120
}
{
  title: 'Digital Marketing Mastery 2026',
  price: 45
}
{
  title: 'Cloud Computing Bootcamp',
  price: 60
}
}
```

3.3 Update Operations (Update)

3.3.1 updateOne() + \$set

Use the updateOne() and \$set command to update the participants info who's name is Ricky, then update his phone number.

```
> db.participants.updateOne(
  { "firstName": "Ricky" },
  { $set: { "phone": "012-3344556" } }
)
```

```
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
_id: ObjectId('695c74edcb0ca891e1a6feeaa')
firstName: "Ricky"
lastName: "Tan"
email: "ricky@gmail.com"
phone: "012-3344556"
gender: "Male"
age: 21
address: "Selangor"
```

3.3.2 updateMany() + \$inc

Use the updateMany() and \$inc command to increase capacity of multiple events where the category is “Technology” by 50 simultaneously.

```
> db.events.updateMany(
  { "category": "Technology" },
  { $inc: { "capacity": 50 } }
)
```

```
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
```

```
_id: ObjectId('695abc9cf084f228b404f029')
title: "Grab Tech Summit 2026"
category: "Technology"
organizerID: ObjectId('695ab30df084f228b404f025')
location: "Kuala Lumpur"
eventDate: 2026-06-15T00:00:00.000+00:00
price: 100
capacity: 150
tags: Array (3)

_id: ObjectId('695c7bb0cb0ca891e1a6fee')
title: "AI & Future Tech Symposium"
category: "Technology"
organizerID: ObjectId('695ab30df084f228b404f025')
location: "Kuala Lumpur"
eventDate: 2026-07-20T09:00:00.000+00:00
price: 120
capacity: 250
tags: Array (3)

_id: ObjectId('695c7bb0cb0ca891e1a6fef0')
title: "Cloud Computing Bootcamp"
category: "Technology"
organizerID: ObjectId('695ab30df084f228b404f025')
location: "Online"
eventDate: 2026-09-05T08:00:00.000+00:00
price: 60
capacity: 550
tags: Array (3)
```

3.3.3 updateOne() + \$push

Use the updateOne() and \$push command to add a new value “Hot Selling” into the tags array of the “Shopee E-Commerce Expo” event.

<pre>> db.events.updateOne({ "title": "Shopee E-Commerce Expo" }, { \$push: { "tags": "Hot Selling" } })</pre>	<pre>< { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }</pre> ▶ _id: ObjectId('695abc9cf084f228b404f02b') title: "Shopee E-Commerce Expo" category: "Business" organizerID: ObjectId('695ab30df084f228b404f026') location: "Johor Bahru" eventDate: 2026-04-20T00:00:00.000+00:00 price: 20 capacity: 300 tags: Array (4) 0: "Business" 1: "E-Commerce" 2: "Expo" 3: "Hot Selling"
---	---

3.3.4 updateMany() + \$pull

Use the updateMany() and \$pull command to remove the specific value “Workshop” from the tags array of all events.

<pre>> db.events.updateMany({ "tags": "Workshop" }, { \$pull: { "tags": "Workshop" } })</pre>	<pre>< { acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2, upsertedCount: 0 }</pre> _id: ObjectId('695c7bb0cb0ca891e1a6fef0') title: "Cloud Computing Bootcamp" category: "Technology" organizerID: ObjectId('695ab30df084f228b404f025') location: "Online" eventDate: 2026-09-05T00:00:00.000+00:00 price: 60 capacity: 550 tags: Array (3) 0: "Cloud" 1: "Coding" 2: "Education" _id: ObjectId('695abc9cf084f228b404f02a') title: "Lazada Online Seller Workshop" category: "Business" organizerID: ObjectId('695ab30df084f228b404f027') location: "Kuala Lumpur" eventDate: 2026-05-10T00:00:00.000+00:00 price: 50 capacity: 70 tags: Array (2) 0: "Business" 1: "E-Commerce"
--	---

--	--

3.4 Delete Operations (Delete)

3.4.1 deleteOne()

Use the deleteOne() command to remove a specific document in “bookings” with specific objectID.

```
> db.bookings.deleteOne({
  "_id": ObjectId("695adc92f084f228b404f080")
})
```

```
< {
  acknowledged: true,
  deletedCount: 1
}
```

```
_id: ObjectId('695adc92f084f228b404f080')
participantId: ObjectId('695acc20f084f228b404f05b')
eventId: ObjectId('695abc9cf084f228b404f029')
bookingDate: 2025-12-01T00:00:00.000+00:00
status: "Confirmed"
payment: Object
```

3.4.2 deleteMany()

Use the deleteMany() command to remove multiple document that match the condition where address is “Melaka”.

```
> db.participants.deleteMany({
  "address": "Melaka"
})
```

```
< {
  acknowledged: true,
  deletedCount: 2
}
```

4.0 Advanced Operations

4.1 Index Creation

4.1.1 Single-field index

Use `createIndex()` to create an index on the `eventId` field in the `bookings` collection to speed up queries that retrieve bookings based on event ID.

```
> db.bookings.createIndex({eventId :1})
```

```
< eventId_1
```

```
> db.bookings.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { eventId: 1 }, name: 'eventId_1' }
]
```

Use `createIndex()` to create an index on the `category` field in the `events` collection to improve query performance when filtering events by category.

```
> db.events.createIndex({category :1})
```

```
< category_1
```

4.1.2 Compound Index

Use `createIndex()` to create a compound index on the `eventId` and `bookingDate` fields to optimize queries that retrieve bookings for a specific event and sort them by booking date.

```
> db.bookings.createIndex({eventId :1, bookingDate : -1})
```

```
< eventId_1_bookingDate_-1
```

```
> db.bookings.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { eventId: 1 }, name: 'eventId_1' },
  {
    v: 2,
    key: { eventId: 1, bookingDate: -1 },
    name: 'eventId_1_bookingDate_-1'
  }
]
```

4.2 Aggregation Pipelines

Use the `$match` stage to filter events that belong to the `Technology` category, followed by the `$group` stage to calculate the average event price and total number of events in that category.

```
> db.events.aggregate([
  { $match: { category: "Technology" } },
  { $group: { _id: "$category", avgPrice: { $avg: "$price" }, totalEvents: { $sum: 1 } } }
])
```

```
< {
  _id: 'Technology',
  avgPrice: 82.5,
  totalEvents: 4
}
```

Use the \$group stage to calculate the total number of bookings for each event, followed by the \$sort stage to rank events based on the number of bookings in descending order.

```
> db.bookings.aggregate([
  {$group: { _id: "$eventId",
             totalBookings: { $sum: 1 } }},
  { $sort: { totalBookings: -1 } }
])
```

```
< {
  _id: ObjectId('695abc9cf084f228b404f02b'),
  totalBookings: 3
}
{
  _id: ObjectId('695abc9cf084f228b404f029'),
  totalBookings: 2
}
{
  _id: ObjectId('695abc9cf084f228b404f02a'),
  totalBookings: 2
}
{
  _id: ObjectId('695abc9cf084f228b404f02c'),
  totalBookings: 2
}
{
  _id: ObjectId('695abc9cf084f228b404f02d'),
  totalBookings: 2
}
```

4.3 Sorting Demonstration

We used ascending order to list events from the lowest price to the highest. This feature helps budget-conscious users identify affordable events quickly.

The query successfully reordered the documents. The event **"Shopee E-Commerce Expo"** (Price: 20) appeared at the top of the list, followed by **"Digital Marketing Mastery"** (Price: 45).

```
db.events.find(
  {},
  { "title": 1, "price": 1, "category": 1 }).sort({ "price": 1 }
).forEach(printjson);
```

```
< 1. ASCENDING SORT: Events sorted by Price (Low to High)
< -----
< {
  _id: ObjectId('695abc9cf084f228b404f02b'),
  title: 'Shopee E-Commerce Expo',
  category: 'Business',
  price: 20
}
< {
  _id: ObjectId('695c7bb0cb0ca891e1a6feef'),
  title: 'Digital Marketing Mastery 2026',
  category: 'Business',
  price: 45
}
< {
  _id: ObjectId('695abc9cf084f228b404f02a'),
  title: 'Lazada Online Seller Workshop',
  category: 'Business',
  price: 50
}
< {
  _id: ObjectId('695abc9cf084f228b404f02c'),
  title: 'Grab Ride & Tech Conference',
  category: 'Technology',
  price: 50
}
< {
  _id: ObjectId('695c7bb0cb0ca891e1a6fef0'),
  title: 'Cloud Computing Bootcamp',
  category: 'Technology',
  price: 60
}
```

We used descending order to analyze booking payments from the highest amount to the lowest. This is useful for administrators to track significant revenue sources or VIP bookings.

The system listed the highest transactions first. The booking with **RM 100** (Booking ID ending in ...085) appeared first, followed by the booking with **RM 80**.

```

db.bookings.find(
  { "status": "Confirmed" },
  { "payment.amount": 1, "bookingDate": 1, "status": 1 }).sort({ "payment.amount": -1 })
).forEach(printjson);

```

```

-----
< 2. DESCENDING SORT: Bookings sorted by Payment (High to Low)
< -----
< {
  _id: ObjectId('695adc92f084f228b404f085'),
  bookingDate: 2025-12-06T00:00:00.000Z,
  status: 'Confirmed',
  payment: { amount: 100 }
}
< {
  _id: ObjectId('695adc92f084f228b404f08a'),
  bookingDate: 2025-12-11T00:00:00.000Z,
  status: 'Confirmed',
  payment: { amount: 100 }
}
< {
  _id: ObjectId('695adc92f084f228b404f084'),
  bookingDate: 2025-12-05T00:00:00.000Z,
  status: 'Confirmed',
  payment: { amount: 80 }
}
< {
  _id: ObjectId('695adc92f084f228b404f089'),
  bookingDate: 2025-12-10T00:00:00.000Z,
  status: 'Confirmed',
  payment: { amount: 80 }
}
< {
  _id: ObjectId('695adc92f084f228b404f081'),
  bookingDate: 2025-12-02T00:00:00.000Z,
  status: 'Confirmed',
  payment: { amount: 50 }
}

```

4.4 Explanation of Performance Improvements

In this project, we optimized query performance by implementing strategic Indexing to avoid inefficient Collection Scans, where the database must read every single document to find a match. By creating indexes, we enabled the database to perform faster Index Scans. We implemented a single-field index on the category field, which is crucial because users frequently filter events by type, such as "Technology" or "Business". This allows the database to retrieve specific categories without scanning unrelated documents. Additionally, we established a compound index on { location: 1, eventDate: 1 }. This optimizes complex queries where users search for events in a specific city sorted by the nearest date, significantly reducing processing load by handling both filtering and sorting in a single efficient step.

4.5 Challenges Encountered

Throughout the development of the NoSQL backend, we encountered several distinct technical challenges that required specific problem-solving strategies. A primary issue involved script automation. We initially faced syntax errors when using the shell helper command `use Event_Booking_System` inside our .js files, as the script processor did not recognize this interactive command. We resolved this by adopting the standard JavaScript method `db.getSiblingDB()` to switch databases programmatically. Additionally, we discovered workflow limitations within MongoDB Compass, where the embedded shell triggered a `MongoshUnimplementedError` when we attempted to use the `load()` function to run external scripts. This forced us to adapt our testing methodology by manually executing code blocks. Furthermore, maintaining data integrity proved challenging without SQL-style foreign keys so we had to verify manually that every `eventId` referenced in the bookings collection corresponded to a valid document in the events collection to effectively link our data.

5.0 Security and Limitations

5.1 Basic Access Controls

To secure the Event Booking System, we must implement Role-Based Access Control (RBAC) rather than allowing anonymous access, which leaves the database vulnerable. This involves creating a specific database administrator, such as eventAdmin, with a secure password and assigning the readWrite role strictly for the Event_Booking_System database. This ensures the application can effectively manage bookings and event details without granting unauthorized access to system configurations or other sensitive databases on the server.

5.2 Injection Risk

Although MongoDB is immune to traditional SQL injection, it is vulnerable to NoSQL Injection, where attackers attempt to manipulate queries by injecting malicious objects. For example, an attacker could send a MongoDB operator like { "\$ne": null } (not equal to null) into a login field to bypass authentication checks. To prevent this risk in our system, strict input sanitization must be implemented at the application level to ensure all user inputs are treated as literal strings rather than executable query objects.

5.3 Database Constraints or Limitations

We identified specific design limitations in our NoSQL implementation compared to traditional relational databases. First, MongoDB does not enforce referential integrity or support cascading deletes. Consequently, if an organizer cancels an event and the document is removed, the associated bookings in the bookings collection remain as "orphaned" records referencing a non-existent eventId, requiring manual cleanup logic in the application code. Second, MongoDB enforces a strict 16MB limit per BSON document. While our current dataset is small, utilizing an embedded data model such as storing all participant details directly within an event document could eventually exceed this limit for large-scale events. Finally, ensuring consistency across multiple collections is more complex. Operations like reducing event capacity while simultaneously creating a booking require resource-intensive multi-document transactions to prevent data inconsistencies such as overbooking.

6.0 Teamwork

Poh Lok Yee – Database Designer

As the database designer, I was responsible for designing the overall NoSQL database for the Event Booking System. This included selecting the event booking system as the project case study and identifying the required collections such as organizers, events, participants, and bookings. I designed the structure of each collection, created sample documents, and ensured that appropriate fields and data types were used to accurately represent real-world event booking scenarios. I also incorporated arrays, embedded documents, and references to ensure that the data was logically connected, such as linking bookings to participants and events. In addition, the sample data was designed to be realistic and meaningful to support CRUD operations and advanced MongoDB queries in later stages of the project.

Lee Yin Shen – CRUD Specialist

As the CRUD Specialist, my main responsibility is to handle the CRUD operation, which includes Create, Read, Update and Delete. I started by inserting data into the database using operator like `insert()` and `insertMany()` so we had enough data to test the system. Then I wrote specific search queries using filters like `$gt` and `$or` to prove our system could “answer questions”, like finding the events with cheaper price. I also handled data updates, like using `$inc` to automatically adjust ticket capacity when a booking is made and `$set` to modify user details. Lastly, I tested delete commands to remove cancelled bookings to ensure the database remained clean and accurate.

Cheryl Cheong Kah Voon – Data Analyst

My role in this project focused on optimizing data access and supporting analytical queries within the Event Booking System. I implemented indexing on key fields to enhance query efficiency and developed aggregation pipelines to generate meaningful summaries from event and booking data. These aggregations were used to analyze patterns such as event pricing by category and booking distribution across events. The implemented solutions were designed to work effectively with the existing database structure and to support performance-oriented data analysis requirements.

Lau Yee Wen- Database Administrator

As the Database Administrator, I focused on the system's performance and security. I implemented the sorting functions to organize events and bookings, such as listing items by price or payment amount. I also analyzed and explained how our indexes improved search speeds compared to normal scans. Additionally, I documented the technical challenges we faced, like script errors and software limitations. Finally, I covered the security section by identifying risks like NoSQL injection and explaining the limitations of MongoDB regarding data consistency.

7.0 Conclusion

This project provided valuable experience in designing and implementing a NoSQL database for an Event Booking System using MongoDB. Through this project, the team gained a deeper understanding of NoSQL data modeling, CRUD operations, indexing strategies, and aggregation pipelines for data analysis. These components demonstrated how MongoDB can efficiently manage and analyse event-related data.

One of the main challenges faced during the project was designing a data structure that balances flexibility and performance. Deciding where to apply indexing and selecting suitable aggregation stages required careful consideration to ensure efficient queries without unnecessary complexity. Ensuring consistency between collections and references was also a challenge during implementation.

For future improvement, the system can be enhanced by adding more advanced aggregation pipelines, implementing additional compound indexes based on real query patterns, and incorporating data validation and security features. Performance testing with larger datasets can also be conducted to further evaluate and optimize the database design.